

Article Domain Tagger

Project Report & Technical Architecture

What it does: Classifies news articles into one of 11 topic domains using a lightweight AI model that runs on a laptop CPU.

1. The Goal

Build a news article classifier that:

- Sorts articles into **11 political/news topic categories**
- Runs under **1 GFLOP** of computation (so it is fast on any CPU)
- Is accurate enough to be useful (target: ~80% macro-F1)

2. The Dataset

Source: `mdonigian/iab-news-classification` from HuggingFace (~106K articles)

Why this dataset? The original plan used a HuffPost dataset with 8 vague categories (like "World" which was too broad). I switched to the IAB (Interactive Advertising Bureau) dataset because it has cleaner, more specific labels that align with real-world news taxonomy.

After filtering: 65,083 articles mapped to 11 labels (41K dropped because they didn't fit my categories).

3. The 11 Labels

The system categorizes articles into the following specific domains. Note that Environment articles were extracted from the Science category using keyword filtering (e.g., "climate", "pollution", "renewable energy").

#	Label	What it covers	Example
0	Politics & Government	Elections, legislation, policy, political figures	"Senate passes infrastructure bill"
1	War & Conflicts	Armed conflict, military operations, terrorism	"Ukraine forces advance in eastern front"
2	Crime & Justice	Criminal acts, law enforcement, courts	"Man arrested for armed robbery"
3	Law & Legal	Civil law, lawsuits, court rulings, regulation	"Supreme Court hears antitrust case"
4	Economy & Business	Markets, finance, companies, trade	"Stock market rallies on earnings"
5	Science & Technology	Research, space, tech products, AI, computing	"Scientists discover new solar cell material"
6	Health	Medical conditions, treatments, pandemics	"WHO warns of disease outbreak"
7	Education	Schools, universities, curricula, admissions	"University announces free tuition"
8	Disasters & Emergencies	Natural disasters, accidents, rescue ops	"Hurricane makes landfall"
9	Sports	Competitive events, teams, athletes	"Team wins championship"
10	Environment & Climate	Climate change, pollution, conservation	"Arctic ice melts at record rate"

4. The Model Architecture

The Problem: Size vs Speed

A standard DistilBERT model has 67M parameters and uses ~10.87 GFLOP per prediction. That's 10x over my budget.

The Solution: Knowledge Distillation

I trained a much smaller student model (4 layers, 256 hidden size, ~11M parameters) by having it learn from the larger teacher model.

How Knowledge Distillation works:

1. **Train a teacher** (DistilBERT, 67M params) on the data -> gets ~80.7% macro-F1.
2. **Cache the teacher's "soft" predictions** (not just the final answer, but the probability distribution over all 11 labels).
3. **Train a student** (4L-256H, 11M params) using both:
 - The soft targets from the teacher (KL divergence loss).
 - The hard labels from the data (cross-entropy loss).
4. **Result:** A model that's 6x smaller but retains most of the accuracy.

Student Architecture

```
Student 4L-256H - Layers: 4 (vs DistilBERT's 6) - Hidden size: 256 (vs 768) -  
Attention heads: 4 (vs 12) - Parameters: 11.2M (vs 67M) - Model size: ~44MB (vs ~268MB)
```

5. Training Process

Step 1: Dataset Preparation (local machine)

- Loaded 106K articles from HuggingFace.
- Filtered to 11 IAB categories.
- Extracted Environment articles from Science via keyword matching.
- Created stratified 80/10/10 train/val/test split (seed 42 for reproducibility).
- Final totals: 52,065 train / 6,509 val / 6,509 test.

5. Training Process (Continued)

Step 2: Teacher Training (Google Colab T4 GPU)

```
Model: distilbert-base-uncased -> 11-label head Epochs: 3 Learning rate: 2e-5  
Batch size: 32 Class weights: inverse frequency (to handle imbalance)
```

Teacher Results:

Epoch	Loss	Accuracy	Macro F1
1	0.6637	82.8%	0.779
2	0.3737	85.1%	0.803
3	0.2565	85.3%	0.807

Step 3: Knowledge Distillation (Google Colab T4 GPU)

```
Student: 4L-256H (11.2M params) Temperature: 3.0 Alpha (KL weight): 0.7 Learning  
rate: 5e-4 Epochs: 5 Seeds: 42 and 1337 (for variance reporting)
```

Student Results:

Seed	Best Val Macro F1
42	0.765
1337	0.777
Mean	0.771 (sd=0.006)

6. Final Test Set Results

82.4% OVERALL ACCURACY	0.766 MACRO F1 SCORE	0.642 GFLOP (PASS < 1.0)
~44 MB MODEL SIZE	~5.8ms CPU LATENCY (P50)	

Per-Class Performance

Label	F1 Score	Strength
Sports	0.958	Excellent
Health	0.870	Very good
Politics & Government	0.866	Very good
Crime & Justice	0.837	Good
Economy & Business	0.818	Good
Disasters & Emergencies	0.786	Good
War & Conflicts	0.772	Decent
Science & Technology	0.723	Moderate
Environment & Climate	0.663	Moderate
Education	0.601	Weak (155 examples)
Law & Legal	0.537	Weakest (221 examples)

Observation: The weakest classes (Education, Law & Legal) are also the smallest ones. More training data for these would improve performance significantly.

7. Why Knowledge Distillation Matters

Comparison	DistilBERT Teacher	Student 4L-256H
Parameters	67M	11.2M
Model size	268 MB	44 MB
FLOPs @96	8.16 GFLOP	0.64 GFLOP
Macro F1	0.807 (val)	0.766 (test)
Runs on CPU?	Slow	Fast (~6ms)

The student retains **95% of the teacher's accuracy** while being **12x more efficient** computationally.

8. Key Design Decisions

- 11 labels instead of 8:** The original 8-label taxonomy had a vague "World" category. The 11-label IAB taxonomy separates War, Law, and Disasters into their own clearer categories.
- max_length = 96 tokens:** Analysis showed 99.7% of articles fit within 96 tokens (p99 = 85). Using 128 wastes compute; using 64 loses too much text context.
- Inverse-frequency class weights:** The smallest class (Education, 1.3%) is ~25x smaller than the largest (Politics, 25.9%). Without weighting, the model would ignore rare classes entirely during training.
- 2-seed reporting:** Running with seeds 42 and 1337 shows the variance between training runs (sd = 0.006), giving statistical confidence in the final results.
- IAB dataset over HuffPost:** The IAB dataset has more specific, internationally-relevant categories and avoids the problematic catch-all categories.

9. Project File Structure

```

    article-domain-tagger/ |— config.py # All shared settings (labels, paths) |—
REPORT.md # This file |— CHECKLIST.md # Progress tracking | |— data/ | |—
load_iab_dataset.py # Downloads & filters IAB dataset | |— taxonomy.py # 11 label
rules, definitions | |— remap_dataset.py # (legacy) re-maps HuffPost categories | |—
split_dataset.py # (legacy) creates stratified splits | |— train/ | |— common.py #
Shared helpers (dataset, loaders) | |— teacher_train.py # Train DistilBERT teacher |
|— cache_teacher_logits.py # Cache predictions for KD | |— train_student.py # Train
student with KD | |— student.py # Student architecture (4L-256H) | |— eval/ | |—
evaluate.py # Test set evaluation | |— deploy/ | |— model_export.py # Export model
with label config | |— analysis/ | |— token_stats.py # Token length distribution |
|— measure_flops.py # FLOP measurement | |— generated/ # Generated data & checkpoints
| |— split_train.csv | |— split_val.csv | |— split_test.csv | |— deployed_student/
# Exported student model | |— student_4L_256H_seed1337/ | |— app.py # FastAPI backend
|— frontend/ # React web interface
```

10. How to Use

Run the web interface:

```
# Terminal 1: Start the API server venv\Scripts\python.exe -m uvicorn app:app --
reload --port 8000 # Terminal 2: Start the web interface cd frontend npm install npm
run dev
```

Open `http://localhost:5173` in your browser. Paste any news article text and click "Classify" to see the prediction.

Run from Python:

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer import
torch model_path = "generated/deployed_student" tokenizer =
AutoTokenizer.from_pretrained(model_path) model =
AutoModelForSequenceClassification.from_pretrained(model_path) model.eval() text =
"Scientists discover high-efficiency solar cell material" inputs = tokenizer(text,
return_tensors="pt", truncation=True, max_length=96) with torch.no_grad(): probs =
torch.softmax(model(**inputs).logits, dim=-1) pred = probs.argmax(dim=-1).item()
print(f"Predicted: {model.config.id2label[pred]} ({probs[0][pred]:.1%})")
```

11. Known Limitations & Future Work

1. **Law & Legal (F1=0.54) and Education (F1=0.60)** are weak due to small dataset size. The project has 42K "ambiguous" rows in `review_ambiguous.csv` that could be manually labeled to boost these classes.
2. **No weighted loss in the original teacher training** means the teacher itself may underperform on rare classes. Adding focal loss could help.
3. **Environment articles** are keyword-extracted from Science, which may introduce noise. A dedicated Environment training corpus would improve this class.
4. **Future improvements:**
 - Manually review the 42K ambiguous rows to add more training data.
 - Try different temperatures ($T=5$) and alpha values for KD.
 - Consider ONNX export for even faster inference.
 - Add a confidence threshold (reject if max probability < 50%).